Programación funcional.

La programación funcional no es más que un paradigma de programación, es decir, es una forma en el cual podemos resolver diferentes problemáticas.
Cuando nos encontramos desarrollamos software utilizando este paradigma, estaremos trabajando con funciones, evitaremos los datos mutables, así como el hecho de compartir estados entre fonciones.
Con este paradigma las funciones serán tratadas como ciudadanos de primera clase. Las funciones podrán ser asignadas a variables además podrán ser utilizadas como entrada y salida de otras fonciones.

A las funciones que puedan tomar funciones como parámetros y devolver funciones como resultado serán conocidas como función de orden superior.

La programación funcional es un paradigma declarativo. Nos enfocaremos en "que" estamos haciendo y no en "como" se está haciendo que sería el enfoque imperativo. Esto quiere decir que nosotros expresaremos nuestra lógica sin describir controles de flujo; no usaremos ciclos o condiciones.

RMAZH

Scanned with CamScanner

CÁLCULO LAMBDA DE ALONSO CHURCH

El cálculo lambda (cálculo λ), creado originalmente por Abnzo Church, es el lenguaje de programación más pequeño del mundo. A pesar de no tener números, cadenas de caracteres, booleanos ni ningún otro tipo de dato que no sea una función, ¡el cálculo lambda puede usarse para representar cualquier máquina de Turing!

El cálculo lambda se compone de 3 elementos variables, funciones y aplicaciones.

Variable <name> x "x" Una variable llamada.

Función λ<parameters>.<body> $\lambda x.x$ Con parámetro "x" y cuerpo "x".

Solicitud <Funtion> <Variable or function> $(\lambda x.x)a$ "$\lambda x.x$" con el argumento "a".

La función más básica es la función identidad: $\lambda x.x$ que es equivalente a $f(x) = x$. La primera "x" es el argumento de la función, y la segunda es el cuerpo de la función.

Variables libres frente a variables ligadas.
En la función $\lambda x.x$ "x" se denomina variable ligada porque se encuentra tanto en el cuerpo de la

Scanned with CamScanner

función como en un parámetro.
En $\lambda x.x$, "y" se denomina variable libre porque nunca se declara de antemano.

Scanned with CamScanner

ELIXIR.

Creado en 2011 por José Valim, está construido sobre Erlang y comparte sus mismas abstracciones combinando la robustez de este con una sintaxis moderna y expresiva inspirada en Ruby y Python.

Utiliza datos inmutables y funciones puras sin efectos secundarios.

Elixir es una tecnología bastante nueva, sin embargo, esto no debería engañarnos, que sea un lenguaje nuevo no significa que sea frágil o que existan bugs. De hecho, Elixir se distingue por ser estable y robusto, principalmente porque los programas creados en este lenguaje se ejecutan en la máquina virtual de Erlang (BEAM)

Elixir se ejecuta sobre la máquina sobre la máquina virtual de Erlang, esto le hereda décadas de robustez en sistemas que no pueden permitirse caerse nunca.

Scanned with CamScanner

ERLANG

Es un lenguaje de programación funcional y un ecosistema de ejecución diseñado específicamente para construir sistemas distribuidos, altamente concurrentes y tolerantes a fallos.

Fue creado en 1986 en los laboratorios de la compañía de telecomunicaciones sueca Ericsson por los ingenieros José Armstrong, Robert Virding y Mike Williams. Su propósito original era resolver un problema crítico de la industria:

Como programar la proxima generación de centrales telefonicas, las cuales requerían manejar millones de llamados simultáneas y operar de forma continua, sin interropciones por mantenimiento o errores de software.

Erlang esta diseñado para resolver problemas que involucran concurrencia, distribución y tolerancia a fallos. A Erlang utiliza un modelo de concurrencia basado en actores, donde los procesos son completamente independientes y se comunican entre sí mediante el paso de mensajes. Los procesos de Erlang son extremadamente ligeros y pueden correr millones de ellos simultáneamente, lo que lo hace ideal para sistemas que requieren manejar múltiples conexiones concurrentes

Una de las principales razones por las que se diseñó Erlang fue para construir sistemas robustos que pudieran seguir funcionando incluso si

Scanned with CamScanner

algunas de sus partes fallaban.

DONDE SE OTILIZAN EN WHATSAPP.

WhatsApp utiliza Erlang de forma masiva en la mayor parte de su código de servidor y en su infraestructura principal, mientras que Elixir que comparte la misma maquina virtual BEAM se ha explorado o utilizado de forma más especifica o en transición hacia Erlang.

Donde se utiliza Erlang en WhatsApp

Infraestructora de Servidores: La práctica totalidad del backend y la lógica de los servidores de WhatsApp esta escrita en Erlang.

Gestión de Conexiones Simoltáneas: Maneja millones de conexiones TCP y conversaciones en tiempo real de forma paralela.

Alta disponibilidad: Aprovecha la tolerancia a fallos y la capacidad de recargar código en caliente sin interrompir el servicio ni generar tiempo de inactividad.

Clústeres y protocolos de distribución.
Utiliza componentes internos como el filtro erldist_filter_nif para supervisar, controlar y asegurar el

Scanned with CamScanner

tráfico de red entre los nodos masivos de Erlang.

El papel de Elixir

Relación tecnológica: Elixir se ejecuta sobre la misma máquina virtual de Erlang (BEAM) y comparte sus mismas ventajas de concurrencia, aunque WhatsApp se ha mantenido históricamente fiel y centrado de forma principal en Erlang nativo.

RMAZH

Scanned with CamScanner

FUNCIONES DE ORDEN SUPERIOR

Las fonciones de orden superior (map, filter y fold/redoce) son herramientas clave de la programación foncional que permiten procesar colecciones o listas de datos de forma declarativa, sin necesidad de escribir bucles for o while explicitos.

1. MAP (Transformar)

Aplica una función a cada uno de los elementos de una lista y devuelve una nueva lista (o on iterador) con los mismos elementos pero transformados. El tamaño de la lista de salida es idéntico al de la lista de entrada.

Qué recibe: Una función y una lista/iterable.
Ejemplo conceptual: map (f, [1, 2, 3] → Aplicar f a cada número → [f(1), f(2), f(3)].

Ejemplo práctico

Python

```
numeros = [1, 2, 3, 4]
dobles = list (map (lamda x: x* 2 , n
  #resultado: [2, 4, 6, 8]
```

2. FILTER (filtrar).

Evalúa cada elemento de una lista mediante una función (predicado) que devuelve true o false.

Solo se conservan aquellos elementos que cumplen con la condición. El tamaño de la lista resultante es igual o menor al original.

Recibe una función booleana (predicado) y una lista/iterable.

Ejemplo: filter(es_par, [1, 2, 3, 4]) → [2, 4]

Ejemplo Práctico.

Python

```
números = [1, 2, 3, 4]
pares = list(filter(lambda x: x % 2
# resultado: [2, 4]
```

3. FOLD / REDUCE (Reducir o acumular).

Combina todos los elementos de una lista uno a uno utilizando una función acumuladora para reducirlos a un único valor final (un número, un objeto, otra lista, etc)

Recibe una función binaria y un iterable

Devuelve un único valor resultante (un número, un string, un objeto acumulado)

Ejemplo. Sumar todos los elementos de [1, 2, 3, 4]

acumula((((1+2)+3)+4) = 10

Python

```
from functools import reduce
numeros = [1, 2, 3, 4]
Suma total = reduce(lambda acumulador, actual: acumulador + actual, numeros)
# resultado: 10
```

MAZH

Scanned with CamScanner

Lizbeth Hernández García.

CARACTERÍSTICA	NOTACIÓN PREFIJA	INFIJA	POSTFIJA.
Posición de la función.	Antes de los argumentos.	Entre los argumentos (o expresiones)	Despues del argumento
Sintaxis general	funcion arg1 arg2	arg1 Operador arg2	arg operador
Soporte nativo en Haskell.	Nativo y principal es la forma estandar de evaluación	Nativo (tanto para operadores simbolicos como f como para fonciones con backticks.	No nativo para funciones generales (requiere extensiones de GHC o simulación manual).
Ejemplo clásico	div 10 20 (+) 53 /10 2	10 `div` 2 05 + 3 5+3	Expresiones RPN (Reserve Polish Notation) evaluadas mediante pilas. 8 9 +
Asociatividad y precedencia	Se evalua estrictamente de izquierda a derecha.	Se puede configurar explicitamente (infixl, infixr, infix) con niveles de prioridad (0-9)	Depende de las extensiones de GHC o de algoritmos de procesamiento de listas.
Cómo convertirla	Encerrando un operador infijo entre paréntesis: (+)	Encerrando una función prefijo entre acentos graves	Requiere algoritmos especificos para parsear cadenas de texto

Scanned with CamScanner

division.hs.

2.3 INTERVALOS.

Los intervalos tambien conocidos como ranges o rangos se generan utilizando la sintaxis de aristas [inicio.. fin]. Esta es ona forma muy conveniente y legible de crear listas de elementos secuenciales.

Intervalos básicos (sintaxis elemental)
Para crear una lista secuencial, simplemente indicas el principio y el final separados por dos puntos.

- [1..10] genera [1,2,3,4,5,6,7,8,9,10]
- ['a'..'z'] genera el abecedario en minúsculas:
['a','b','c',...,'z']

Definir un paso.
Si no quieres que avance de uno en uno, puedes especificar el segundo elemento de la serie antes de los dos pontos. Haskell calculará la diferencia (el paso) automáticamente.

- [2,4..10] genera los números pares: [2,4,6,8,10] (el paso es +2)
- [10,9..1] genera una cuenta regresiva:
[10,9,8,7,6,5,4,3,2,1] (el paso es -1).

Intervalos infinitos
Debido a que Haskell utiliza evaluación perezosa puedes

Scanned with CamScanner

crear rangos que no tienen un límite final. El programa solo calculará los números que necesites en el momento.
[1..] genera la lista infinita de números naturales
[1,2,3,4,5,...]

[5,10..] genera los multiplos de 5 de forma infinita
[5,10,15,20,...]

Para usarlos sin que el programa se quede colgado en un bucle infinito se suelen combinar con funciones como take.

Haskell
take 5 [1..].

2.4 OPERADORES.

Un operador es fondamentalmente una función que uno o dos argumentos Caridad 102). La diferencia principal frente a las funciones tradicionales con nombres de letras (como div o mod) radica en su sintaxis de uso y en las reglas estrictas que rigen su precedencia y asociatividad.

Funciones Infijas Prefijas.
Por defecto, los operadores en Haskell otilizan notación por infija.

Scanned with CamScanner

Haskell
resultado = 5+3 — Notación infija estandar.

Sin embargo cualquier operador infijo puede convertirse en una función prefija (donde el nombre del operador va al principio y recibe ambos argumentos) si se encierra entre parentesis ():

```
resultado = (+) 5 3 -- equivale a 5+3
```

Por el contrario, cualquier función alfanúmerica normal de dos argumentos (como div) puede utilizarse como un operador infijo si se encierra entre acentos graves ``:

```
resultado = 10 `div` 2 -- Equivalente a div 10 2
```

Haskell permite aplicar parcialmente un operador infijo utilizando on solo operando, ya sea a la izquierda o la derecha, encerrando la expresión entre paréntesis.
Esto genera fonciones altamente legibles

Sección derecha (op x): Equivale a \y -> y op x

```
Haskell
multiplicarPorDos = (* 2)
-- multiplicarPorDos 5 => 10.
```

Scanned with CamScanner

2.7 Evaluación perezosa

Es el comportamiento predeterminado (default). A diferencia de la mayoría de los lenguajes imperativos o funcionales estrictos (como Java, Python o C++), Haskell no evalúa una expresión en el momento en que se define o se pasa como argumento, sino únicamente cuando su resultado es indispensable para producir el valor final que se va a mostrar o procesar.

Como funciona bajo el capó (Thunks)

Cuando defines una expresión en Haskell, el sistema no calcula el resultado inmediatamente; en su lugar, crea un objeto interno llamado thunk

Creación: El thunk se crea y se pasa por el programa sin resolverse.

Evaluación: Cuando una función externa (como print, (+) o un patron de coincidencia) exige conocer el valor concreto, el thunk se evalua.

Memorización: Una vez evaluado, el thunk es reemplazado por su valor calculado. Si se vuelve a invocar esa misma variable o expresión, nunca se vuelve a recalcular, se devuelve directamente el valor almacenado.